Building and Breaking a Raft Key-Value Store

Design, 24 bugs, and what found them

Technical report on raftkv, a replicated key-value store written from scratch in Java 25 with no runtime dependencies. Code and README: see the repository this file lives in.

---

1. Summary

raftkv is a small distributed key-value store. A cluster of nodes agrees on every write using the Raft consensus algorithm, keeps serving while a minority of nodes fail, and never loses a write it has acknowledged. It has leader election (with PreVote and CheckQuorum), log replication, durable storage, snapshots with log compaction, retry-safe client writes, linearizable reads, a live web dashboard, a benchmark tool and a Docker Compose setup.

This report is not a claim of novelty: Raft is well understood and every algorithmic feature here comes from the original paper or the author's thesis. It is an engineering report about three things:

1. **What the design looks like**, including the decisions the paper leaves open.
2. **Which bugs the implementation actually had**, and how each was found. There were 24 worth writing down, and the interesting part is that different testing techniques found very different kinds of bug.
3. **What the system does and does not do well**, with measurements: about 4,650 linearizable reads per second, about 620 writes per second with 16 clients (3.3 times better after adding group commit), and a leader failover in roughly a second, all on a laptop.

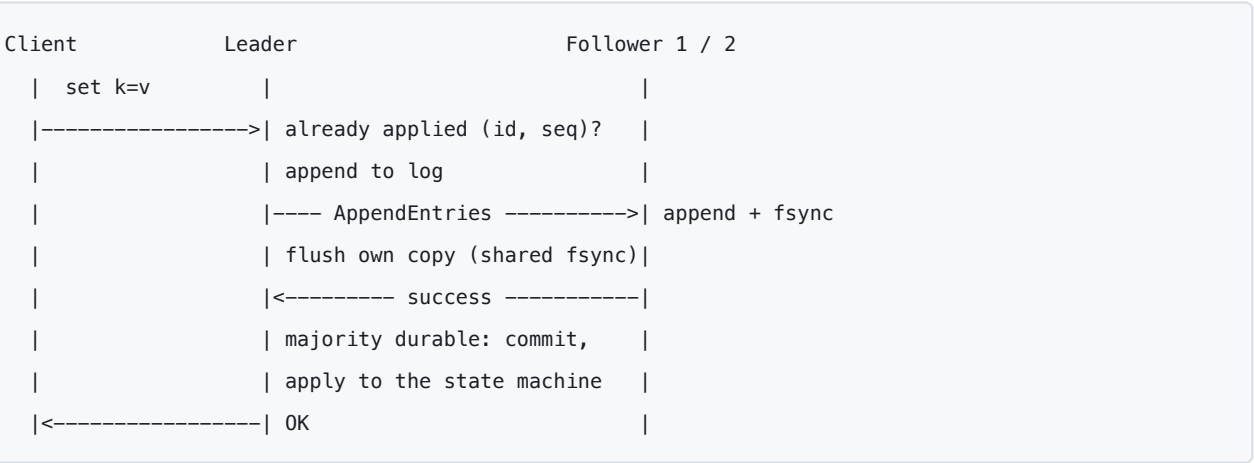
The main lesson is the one distributed-systems people repeat: the algorithm fits on a page, and making it survive failures is the real work.

2. System overview

Each node is a single Java process serving HTTP and JSON:



A client talks to the leader. A write is appended to the leader's log, replicated to followers, and acknowledged once a majority has it durably:



Reads also go through the leader. Before answering, it checks that it has committed an entry from its own term and confirms with a heartbeat round that a majority still follows it.

3. Design decisions

The Raft paper specifies the core protocol. These are the places where an implementation has to decide something.

One no-op per term. When a node becomes leader it appends a no-op entry. The paper's commit rule only lets a leader commit entries from its own term, so without a new entry from the new term, older entries could stay uncommitted. The no-op also tells the leader when its state machine is current, so reads wait for it.

Reads confirm leadership. A leader that has been cut off can still believe it leads. To avoid serving stale data, each read triggers a heartbeat round and is answered only if a majority replies in the leader's term. This costs a network round trip but no disk write, which is why reads are about seven times faster than writes here.

Exactly-once writes through sessions. If a write times out, the client does not know whether it committed. Retrying blindly could apply it twice or overwrite a later write from another client. Each client therefore sends a client ID and a sequence number, and every replica remembers the highest sequence applied per client and ignores anything at or below it. The table is rebuilt from the log on restart and is part of every snapshot.

Snapshots with a base offset. After a configurable number of applied entries, a node writes its state to a snapshot file and compacts the log. The log keeps entries after a base index, and the base index and term come from the snapshot. The snapshot is saved before the log is compacted, so a crash between the two steps just leaves entries the snapshot already covers, which startup discards. A follower that needs entries that were compacted away receives the whole snapshot in one `InstallSnapshot` request.

PreVote and CheckQuorum. A node that cannot hear a leader normally starts an election and raises its term, which deposes a healthy leader when the node rejoins the cluster. PreVote makes the node first ask whether others would vote for it, without changing anyone's term. Voters refuse if they heard from a leader recently ("leader stickiness"), so a node that can still reach its leader gets refused. CheckQuorum is the other half: a leader that has not heard from a majority for 1.5 seconds steps down, so clients are redirected quickly instead of waiting for writes that cannot commit.

Group commit. Every log append needs an fsync, and an fsync is slow (3.7 ms on the test machine). The leader appends a client write to its log without syncing, releases the node lock, then waits for a flush that covers every write appended since the last one. The safety rule that makes this correct is that the leader counts its own copy toward a majority only up to the last durable index. Followers still flush before acknowledging, so "committed" still means "a majority have it on disk".

One request in flight per follower. The leader sends at most one `AppendEntries` to each follower at a time. Writes that arrive meanwhile accumulate and go out together in the next request, so the follower pays one fsync for many entries. Without this, each client write produced its own request carrying a few new entries, each costing a follower fsync under its lock.

4. The bugs

The implementation had 24 bugs worth recording. They are listed in full in the appendix. This section tells the stories that taught the most, grouped by how they were found.

4.1 Found by integration tests: concurrency

A lock held across a network wait. The first cluster test that wrote a key failed: every write returned "timed out waiting for majority replication" after two seconds. The write handler was `synchronized` and waited for the commit while holding the node's lock, but committing required the replication threads to take that same lock to record follower replies. Nothing could proceed until the two-second timeout released the lock, after which the write did commit, so clients were told it failed even though it succeeded. The held lock also stalled heartbeats, so followers timed out and started elections: the state files left over from earlier runs showed a term number of 57, consistent with constant elections. The fix was removing one keyword. The lesson: concurrency bugs in distributed code often look like network problems.

A timer that fires after it was reset. A flaky test led to tracing elections, which showed a node starting an election 12 milliseconds after it had granted a vote. A scheduled timer task that has already started cannot be cancelled, so a timeout that fired while a vote request held the node's lock still ran afterwards. The fix is to re-check, under the node's lock, whether the timer has really elapsed since its last reset. The test that found this passed most of the time and failed only now and then, which is exactly how these bugs hide.

4.2 Found by model-based tests and fuzzing: the data layer

Fuzz tests feed random and mutated data to parsers, and model-based tests compare a component to a trivially correct version after random operations. They found three bugs that example-based tests had missed:

- **Deeply nested JSON overflowed the stack.** A request body of 200,000 opening brackets threw `StackOverflowError` in the hand-written parser. The parser now rejects nesting deeper than 100 levels.
- **A crash in the middle of a character made the node unable to start.** The log is one entry per line. Recovery correctly dropped a half-written last line, but it first decoded the whole file as UTF-8, and a write cut inside a multi-byte character made decoding throw. A test that truncates the log at every byte offset found it. Recovery now splits on raw bytes before decoding.
- **An unpaired surrogate silently changed data.** A key containing half of an emoji (valid in JSON and in Java) was stored and acknowledged, but UTF-8 cannot encode it, so it was written to disk as `?`. After a restart the data differed from what had been acknowledged, and different keys could collide. Keys and values must now be well-formed text.

4.3 Found by an independent review: security and robustness

A separate reviewer read all the code without the author's assumptions. It found no safety violation and no deadlock, but it found problems the tests were not designed to look for:

- **Forged Raft messages from a web page.** The client write endpoints required JSON requests, which forces browsers to run a CORS check, but the three node-to-node endpoints did not. A page on any website could send a plain cross-site POST containing a JSON body and make a local node adopt a huge term or install a forged snapshot. The Raft endpoints now accept only JSON POSTs, and a test confirms that a forged request leaves the node's term unchanged.

- **Script injection through the dashboard.** The dashboard printed whatever a node reported into the page. A malicious node, or a link pointing at one, could run script in the dashboard's origin. The page now validates and coerces every field. This was verified in a browser against a fake hostile node.
- **Unbounded work.** Request bodies had no size limit, and a follower far behind received its whole backlog in one request with one fsync per entry.

4.4 Found by the chaos test

The chaos test partitions, crashes and restarts nodes at random while clients write and read. It checks that no term ever has two leaders, no acknowledged write is lost, no read returns a wrong value, and every replica converges once the faults stop. It did not find a bug in the final code, but it was the most valuable test for another reason: it caught a deliberately broken voting rule when that rule was disabled, which is the evidence that the test can fail. The first attempt at this check did not fail, and the reason is instructive: PreVote contains its own copy of the log up-to-date check and masked the broken one. The experiment only worked after both copies were disabled.

4.5 Bugs in the tests themselves

Two failing tests were the tests' fault. One assumed that a write which failed with "timed out waiting for majority" would never take effect. In Raft such a write stays in the leader's log and can commit when the other nodes return, so the guarantee is that every replica agrees on it, not that it is absent. Another asserted a property of a function that is true by design for a case that never occurs. Both are reminders that a test encodes a belief about the system, and the belief can be the wrong part.

5. How it was tested

The suite has about 290 tests in these layers:

Layer	What it checks	Example of what it found
Unit tests	Log, snapshots, state machine, JSON, config	Edge cases in recovery
Model-based tests	A component against a trivially correct model after random operations	Lone-surrogate corruption
Fuzz tests	Random and mutated input to every parser	Stack overflow on deep JSON
Crash tests	The log file cut at every byte offset	Startup failure after a torn multi-byte write
Protocol robustness	Malformed, hostile and cross-site requests to every endpoint	Wrong-typed fields causing HTTP 500
Scenario tests	Injected partitions, crashes and restarts with specific expectations	Cut-off leaders, stale reads, minority partitions
Chaos test	Randomized faults with invariant checks over many seeds	(Nothing in the final code; caught a deliberate bug)
Independent review	Reading the code adversarially	Forged RPCs, injection, unbounded work

Two limits are worth stating. The tests simulate **process crashes**, not **power loss**: a killed process still leaves its writes in the operating system's cache, so a bug in the order of "write, flush, acknowledge" would not show up. The durability ordering, including group commit's rule that the leader counts its own copy only once it is flushed, is argued from the design and not tested against a filesystem that drops unflushed data. And the chaos test is randomized, so a clean run is evidence, not proof: it ran at least 32 twenty-second runs (plus four eight-second runs in every full test run) across three- and five-node clusters without a violation.

6. Performance

Measured on one laptop (Apple M5, 10 cores, JDK 25.0.3). A 3-node cluster and the load generator share one JVM, with real HTTP over loopback and an fsync before an entry counts as stored. Treat the numbers as a relative guide, not a production sizing.

Workload (16 client threads)	Throughput	p50	p95	p99
Writes	618 ops/s	24.7 ms	39.7 ms	52.0 ms
Reads (linearizable)	4,652 ops/s	3.4 ms	4.0 ms	5.7 ms

Clients	1	4	16	64	128
Writes/s	95	225	634	1,097	1,194
p50 latency	11 ms	18 ms	24 ms	57 ms	106 ms

Leader failover (10 trials) took a median of 914 ms to elect a new leader and 919 ms until a client write was accepted again (p95 1.3 s). That is dominated by the randomized 800 to 1500 ms election timeout. A follower that missed 5,000 writes caught up in about 0.1 to 0.2 seconds by either log replay or snapshot.

The group commit result

The first benchmark of the write path was disappointing: 190 writes per second with a median latency of 84.5 ms at 16 clients. Rather than guess, a short probe measured the cost of a single fsync on the test machine:

- One fsync per append: **3.7 ms**, which caps a serial writer at about 270 writes per second.
- Three hundred appends followed by one fsync: **5.9 ms in total**.

That ratio (about fifty to one) says that batching is where the performance is. Two changes followed, both described in section 3: group commit on the leader, then one `AppendEntries` in flight per follower so that followers also batch. The effect at 16 clients:

	Before	After group commit only	After both changes
Writes/s	190	320 to 360	618
p50 latency	84.5 ms	42 to 49 ms	24.7 ms

The benchmark now also reports fsyncs per write: about 0.1 to 0.3 per node at 16 clients, meaning several writes share each flush. The middle column matters as a method point: the first change gave less than the probe promised, and the per-node fsync counts showed why (followers were still flushing once per small request) and pointed at the second change. Throughput keeps rising with more clients but with diminishing returns (1,097 writes per second at 64 clients, 1,194 at 128), because each node can only do so many flushes per second.

7. Limitations and next steps

- **No cluster membership changes.** The set of nodes is fixed at startup. This is the largest missing Raft feature and the riskiest to add, because it is where subtle safety bugs live.
- **No authentication or TLS.** Anyone who can reach a node can read and write data and send Raft messages. The JSON-only rule stops browsers from forging requests; it is not access control.
- **The client session table is never pruned.** It grows with the number of client IDs. Expiring entries deterministically requires a snapshot format change.
- **Snapshots are sent in a single request**, which is fine for state that fits in memory.
- **Durability under power loss is argued, not tested** (section 5).
- **The next step for confidence** is a linearizability checker over recorded chaos-test histories, checking the actual guarantee instead of simpler invariants.

8. Takeaways

1. **The algorithm is the small part.** Most of the bugs lived in locks, timers, file formats, character encodings, input validation and browsers, not in the Raft rules (five of the 24 were genuine protocol-logic errors).
2. **Test invariants, not examples.** The randomized tests that check "one leader per term" and "no acknowledged write is lost" cover scenarios nobody wrote down.
3. **Make the tests able to fail.** Deliberately breaking a rule and confirming that a test notices is cheap, and in this project it revealed a masking effect that would otherwise have given false confidence.
4. **Different methods find different bugs.** Integration tests found races, fuzzing found data-layer bugs, and an independent review found security gaps. None of them alone would have found all 24.
5. **Measure before optimizing, and again after.** A 20-line probe of fsync cost explained the write throughput before any code changed, and the per-node fsync counters showed that the first optimization was only half of the answer.

Appendix: the 24 bugs

#	Bug	Found by	Effect
1	Write handler held the node lock while waiting for commit	Integration test	Every write timed out at 2 s; heartbeats stalled and caused election churn

#	Bug	Found by	Effect
2	Leader answered reads without confirming it still led	Design review	Stale reads from a deposed leader
3	A follower committed up to its log end, not the last entry the leader verified	Review, handler test	Could commit an unverified stale suffix
4	A follower I/O error made the leader retry in a tight loop	Review	Hammering a failing follower
5	`matchIndex` could move backwards on reordered replies	Review	Wrong commit decisions
6	Term/vote and log rewrites were not atomic	Review	Corruption after a crash during a write
7	A torn final log line made the log unreadable	Review	Node could not start after a crash
8	A SET without a value passed validation	Review	Crashed apply on every replica, stalling the cluster
9	A single-node cluster never committed	Review, then a test	No write ever succeeded
10	A stale election-timer callback ran after a reset	Flaky test, tracing	Spurious elections that deposed a healthy leader
11	The client gave up within a second during an election	Review, failover test	Spurious failures during normal failover
12	Deeply nested JSON overflowed the stack	Fuzz test	Handler failure from one hostile request
13	A crash inside a multi-byte character blocked startup	Byte-offset crash test	Node unable to restart
14	Unpaired surrogates were acknowledged, then stored as `?`	Model-based fuzz	Silent data change and key collisions after restart
15	Wrong-typed fields and out-of-range ids caused HTTP 500	Robustness tests	Server errors instead of rejections
16	Invalid UTF-8 in a query merged distinct keys	Robustness tests	Wrong key served
17	Raft endpoints accepted forged cross-site requests	Independent review	A web page could depose the leader
18	No limit on request body size	Independent review	One request could exhaust memory
19	Unbounded `AppendEntries` batches, one fsync per entry	Independent review	A lagging follower stalled and timed out
20	Log truncation changed memory before the durable rewrite	Independent review	Memory and disk could diverge after an I/O error
21	Election timer used a sentinel that overflows with negative `nanoTime`	Independent review	Possible spurious timer firing
22	Dashboard injected node-reported data into the page	Independent review	Script injection; one exception stopped polling
23	Two nodes could share one data directory	Own analysis	Interleaved writes would corrupt the log

#	Bug	Found by	Effect
24	A rejoining or cut-off node raised terms; an isolated leader never stepped down	Test-run traces	Needless elections and slow client failover

---

About this report. The numbers are from the benchmark and test runs described above, on one machine. The project was built with the help of an AI coding assistant (Claude Code), including parts of the implementation, the tests, and an independent code review that found several of the bugs listed above.